

**LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

**MODUL 7
IMPLEMENTASI PROSES DESIGN PROSEDURAL**

**DISUSUN OLEH :
AMMAR BAGAS FATHURRAHMAN WANTORO 2350081008**



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI	i
DAFTAR GAMBAR	ii
BAB I. HASIL PRAKTIKUM	3
I.1 Form Tambah	3
I.1.A. Source Code	3
I.1.B. Hasil	3
I.1.C. Analisa	4
I.2 Simpan data	4
I.2.A. Source code	4
I.2.B. Analisa	5
BAB II. TUGAS PRAKTIKUM	6
II.1 Tampil data	6
II.1.A. Source code	6
II.1.B. Hasil	7
II.1.C. Analisis	8
II.2 Hapus data pesan	8
II.2.A. Source code	8
II.2.B. Analisa	9
BAB III. KESIMPULAN	10

DAFTAR GAMBAR

gambar 1. 1 Tampilan form tambah	3
gambar 2. 1 Tampilan data pesan	7

BAB I. HASIL PRAKTIKUM

I.1 Form Tambah

I.1.A. Source Code

```
<!DOCTYPE html>

<html>

<head>

<title>Tambah Pesan</title>

</head>

<body>

    <form action="simpanPesan.php" method="post">

        <label for="penerima">Penerima</label>

        <input type="number" name="penerima"><br>

        <label for="judul">judul</label>

        <input type="text" name="judul"><br>

        <label for="textPesan">pesan</label>

        <textarea name="textPesan"></textarea>

        <input type="submit" name="submit" value="Simpan">

    </form>

</body>

</html>
```

I.1.B. Hasil



gambar 1.1 Tampilan form tambah

I.1.C. Analisa

Implementasi antarmuka untuk fitur "Tambah Pesan" dibangun menggunakan struktur HTML sederhana yang dirancang untuk memfasilitasi interaksi pengguna dengan sistem secara langsung. Kode program mendefinisikan formulir menggunakan tag <form> dengan atribut action="simpanPesan.php" dan method="post", yang dipilih untuk memastikan keamanan pengiriman data pesan ke server tanpa menampilkan isi data pada URL browser. Elemen input dikonfigurasi sesuai tipe datanya, seperti penggunaan type="number" pada kolom penerima untuk membatasi input hanya berupa angka (ID User), serta tag <textarea> untuk kolom pesan guna menampung teks yang lebih panjang, yang secara efektif meminimalkan kesalahan input data (human error) di sisi antarmuka pengguna.

Dalam konteks desain prosedural dan alur data (DFD), formulir ini berperan sebagai pemicu input untuk proses "6.1 Tambah Pesan". Ketika tombol "Simpan" ditekan, alur program akan beralih secara prosedural ke skrip backend `simpanPesan.php`, yang bertugas menangkap variabel global `$_POST` berisi data penerima, judul, dan isi pesan. Skrip tersebut kemudian akan menjalankan logika penyimpanan data yang menghubungkan input pengguna dengan entitas basis data, merealisasikan aliran data dari entitas "Member" menuju tabel penyimpanan "Pesan" sesuai dengan spesifikasi perancangan sistem yang telah ditetapkan.

I.2 Simpan data

I.2.A. Source code

```
<?php

$idPengirim= 1;

$idPenerima= $_POST['penerima'];

$judul=$_POST['judul'];

$textPesan=$_POST['textPesan'];

$date = date('Y-m-d H:i:s');

$conn = mysqli_connect("localhost","root","", "pesan_db");

$sql      =      "insert      into      data_pesan
(penerima_id, pengirim_id, judul, text_pesan, waktu_pesan)
values ($idPenerima, $idPengirim, '$judul', '$textPesan', '$date') ";

if (mysqli_query($conn, $sql)) {
```

```
header('Location: tampilPesan.php');

echo "Pesan berhasil ditambahkan";

}else{

    echo "Pesan gagal ditambahkan";

}

?>
```

I.2.B. Analisa

Skrip simpanPesan.php berfungsi sebagai backend processor yang menangani logika bisnis penyimpanan data setelah pengguna mengirimkan formulir. Proses dimulai dengan menangkap data inputan yang dikirim melalui metode HTTP POST, yaitu ID penerima, judul, dan isi pesan, serta secara otomatis menetapkan atribut tambahan seperti ID pengirim (ditetapkan statis 1) dan timestamp transaksi saat ini menggunakan fungsi date(). Kode ini kemudian menginisialisasi koneksi ke Database Management System (DBMS) MySQL dengan target basis data pesan_db, mempersiapkan jalur komunikasi agar data transaksi pesan dapat direkam secara permanen.

Pada tahap manipulasi data, skrip menjalankan perintah Data Manipulation Language (DML) berupa kueri INSERT INTO untuk memasukkan variabel-variabel tersebut ke dalam entitas tabel data_pesan. Implementasi struktur kontrol if-else digunakan untuk memvalidasi eksekusi kueri; apabila proses penyisipan data berhasil, fungsi header() akan memicu redirection otomatis ke halaman tampilPesan.php untuk menampilkan pembaruan data secara real-time kepada pengguna. Mekanisme ini menuntaskan siklus alur data pada fitur "Tambah Pesan" dengan memastikan integritas penyimpanan data sebelum mengembalikan antarmuka ke halaman utama.

BAB II. TUGAS PRAKTIKUM

II.1 Tampil data

II.1.A. Source code

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

        <meta name="viewport" content="width=device-width, initial-
scale=1.0">

        <title>Document</title>

</head>

<body>

    <table border="1" cellpadding="5" cellspacing="0">

        <tr>

            <th>Penerima id</th>

            <th>Pengirim id</th>

            <th>Judul pesan</th>

            <th>Isi pesan</th>

            <th>Waktu pesan</th>

            <th>action</th>

        </tr>

        <?php

            $conn = mysqli_connect("localhost","root","","pesan_db");

            $sql = "select * from data_pesan";

            $res = mysqli_query($conn,$sql);

            if (mysqli_num_rows($res) >0) {

                while ($row = mysqli_fetch_assoc($res)) {

                    echo

                        "<tr>
```

```

        <td>".$row['penerima_id']. "</td>

        <td>".$row['pengirim_id']. "</td>

        <td>".$row['judul']. "</td>

        <td>".$row['text_pesan']. "</td>

        <td>".$row['waktu_pesan']. "</td>

        <td>

            <form action='hapusPesan.php' method='get'>

                <input hidden name='id'
value='".$row['pesan_id']."'>

                <input type='submit' name='submit'
value='hapus pesan'>

            </form>

        </td>

    </tr>";

    }

}

?>

</table>

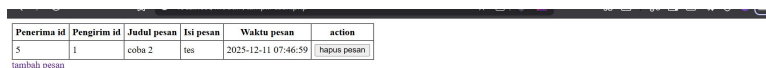
<a href="tambahPesan.php">tambah pesan</a>

</body>

</html>

```

II.1.B. Hasil



Penerima id	Pengirim id	Judul pesan	Isi pesan	Waktu pesan	action
5	1	coba 2	tes	2025-12-11 07:46:59	hapus pesan

[tambah pesan](#)

gambar 2. 1 Tampilan data pesan

II.1.C. Analisis

Kode ini mengimplementasikan antarmuka utama untuk fitur "Lihat Pesan" yang merepresentasikan fungsi Read dalam arsitektur CRUD. Secara teknis, skrip dimulai dengan membangun koneksi ke basis data pesan_db dan mengeksekusi perintah SQL `SELECT * FROM data_pesan` untuk menarik seluruh rekam data yang tersimpan. Hasil kueri tersebut kemudian divalidasi ketersediaannya; jika data ditemukan, program menggunakan struktur perulangan `while` dan fungsi `mysqli_fetch_assoc` untuk mengiterasi setiap baris data. Proses ini secara dinamis mengonversi data mentah dari basis data menjadi elemen baris tabel HTML (`<tr>`), menyajikan informasi krusial seperti ID pengirim, penerima, judul, isi, dan waktu pesan dalam format tabular yang terstruktur dan mudah dibaca oleh pengguna.

Selain fungsi visualisasi data, halaman ini juga bertindak sebagai pusat navigasi untuk manipulasi data lanjutan. Pada setiap baris tabel, terdapat kolom "action" yang memuat formulir khusus yang mengarah ke `hapusPesan.php` menggunakan metode GET. Formulir ini secara cerdas menyisipkan elemen `<input type="hidden">` yang membawa nilai unik `pesan_id`, memastikan bahwa ketika tombol "hapus pesan" ditekan, sistem dapat mengidentifikasi secara spesifik pesan mana yang harus dihapus tanpa menampilkan ID tersebut secara mencolok kepada pengguna. Ditambah dengan adanya tautan menuju `tambahPesan.php`, kode ini menuntaskan alur kerja prosedural dengan menghubungkan fitur pembacaan data, penghapusan, dan penambahan data dalam satu ekosistem antarmuka yang terintegrasi.

II.2 Hapus data pesan

II.2.A. Source code

```
<?php

$idPesan = $_GET['id'];

$conn = mysqli_connect("localhost","root","","pesan_db");

$sql = "delete from data_pesan where data_pesan.pesan_id = '$idPesan'";

if(mysqli_query($conn,$sql)){

    header('Location: tampilPesan.php');

    echo "Pesan berhasil ditambahkan";
```

```
    }else{  
        echo "Pesan gagal ditambahkan";  
    }  
  
?>
```

II.2.B. Analisa

Kode ini mengimplementasikan fungsi Delete dalam operasi CRUD yang bertugas menghapus data spesifik dari basis data. Proses dimulai dengan menangkap variabel id yang dikirimkan melalui URL menggunakan metode \$_GET['id']. Nilai ini merupakan Primary Key (pesan_id) yang menjadi acuan krusial dalam perintah SQL DELETE FROM data_pesan. Penggunaan klausa WHERE yang spesifik pada kueri SQL sangat penting di sini untuk memastikan bahwa operasi penghapusan hanya menargetkan satu baris data yang dimaksud, mencegah risiko terhapusnya seluruh data dalam tabel secara tidak sengaja.

Setelah kueri didefinisikan, skrip mengeksekusinya menggunakan fungsi mysqli_query() yang dibungkus dalam struktur kendali kondisional if. Apabila operasi penghapusan berhasil divalidasi oleh database, fungsi header('Location: tampilPesan.php') akan segera mengalihkan pengguna kembali ke halaman utama, menciptakan alur interaksi yang mulus di mana data yang dihapus langsung hilang dari tampilan tabel. Meskipun terdapat kesalahan teks pada pesan echo (tertulis "berhasil ditambahkan" yang seharusnya "berhasil dihapus"), logika program secara fungsional tetap valid dalam melakukan penghapusan data dan mengelola navigasi pengguna.

BAB III. KESIMPULAN

Praktikum ini telah berhasil mendemonstrasikan penerapan prinsip desain prosedural dalam pengembangan aplikasi web dinamis berbasis PHP dan MySQL. Melalui implementasi fitur pengelolaan pesan, praktikum ini memperlihatkan bagaimana logika program disusun secara linear dan terstruktur, dimulai dari antarmuka input pengguna pada formulir HTML hingga pemrosesan data di sisi server (backend). Pendekatan prosedural ini terbukti efektif dalam memecah alur kerja yang kompleks menjadi serangkaian instruksi logis yang dieksekusi langkah demi langkah, memastikan bahwa setiap variabel input ditangkap, divalidasi, dan diproses sesuai urutan fungsinya sebelum berinteraksi dengan basis data.

Dalam aspek manipulasi data, modul ini sukses mengimplementasikan operasi dasar basis data yang mencakup fungsi Create dan Read. Skrip `simpanPesan.php` membuktikan kemampuan sistem dalam menangani data masuk menggunakan kueri INSERT, sementara `tampilPesan.php` memvisualisasikan data yang tersimpan menjadi informasi yang mudah dibaca menggunakan kueri SELECT dan perulangan. Sinergi antara kedua skrip ini menunjukkan inti dari pemrograman prosedural, di mana fungsi-fungsi terpisah bekerja sama untuk mengelola siklus hidup data—dari saat data diciptakan oleh pengguna hingga ditampilkan kembali dalam format tabular yang rapi.

Terakhir, integrasi fitur penghapusan data melalui `hapusPesan.php` melengkapi fungsionalitas sistem sebagai aplikasi CRUD (Create, Read, Update, Delete) yang utuh. Penggunaan metode pengiriman data melalui URL (GET request) untuk mengidentifikasi target data yang akan dihapus menunjukkan teknik komunikasi antar-modul yang efisien dan cepat. Secara keseluruhan, praktikum ini menyimpulkan bahwa pemahaman mendalam tentang desain prosedural dan konektivitas basis data (mysqli) adalah fondasi krusial dalam membangun sistem informasi yang responsif, di mana setiap komponen kode memiliki peran spesifik namun saling terhubung membentuk satu kesatuan alur kerja aplikasi yang solid.